Filters

Filter is a programm in unix . It takes its input from another program, performs some operation on that input, and writes the result to the standard output. Thus the common use of filters is to modify or restructure output.

Some common filters in UNIX are:

- uniq – Removes identical adjacent lines
- head – displays first n lines of a file .
- tail – displays last n lines of a file .
- sort – sorts files by line (lexically or numerically)
- cut – select portions of a line.
- wc – word count (line count, character count)
- tr – translate
- grep, egrep – search files using regular expressions

head

This command list the beginning of a file to standard output. The default is 10 lines, but a different number can be specified. The command has a number of options.

Syntax:

head [OPTION] [FILE]

Options:

-c Prints the first N bytes of file; with leading -, prints all but the last N bytes of the file.

-n Prints first N lines; with leading - print all but the last N lines of each file.

Example: To display the first 10 lines of the file myfile.txt. $head myfile.txt

To display the first 100 lines of the file myfile.txt.

$head -n100 myfile.txt

To print the first 5 bytes from the file

$ head -c5 myfile.txt

tail

List the (tail) end of a file to stdout. The default is 10 lines, but this can be changed with the -n option. Commonly used to keep track of changes to a system log-file, using the -f option, which outputs lines appended to the file.

Syntax:

tail [OPTION]... [FILE]...

Example:

To display the last 10 lines of the file myfile.txt.

$tail myfile

To display the last 100 lines of the file myfile.txt.

$ tail -100 myfile.txt

$tail –n 100 myfile.txt

more

more command allows to view text files or other output in a scrollable manner. When can command is used to view a very long file, all the output scrolls off the top of your screen and only the last page can be viewed. more command solves this problem by allowing the output of cat command one screenful of data at a time.

Syntax:

more [option] filename

Options:

-num This option specifies an integer which is the screen size (in lines).

-d more will prompt the user with the message "[Press space to continue, 'q' to quit.]" and will display "[Press 'h' for instructions.]" instead of ringing the bell when an illegal key is pressed.

-l more usually treats ^L (form feed) as a special character, and will pause after any line that contains a form feed. The -l option will prevent this behavior.

-p Do not scroll. Instead, clear the whole screen and then display the text.

tr

tr command automatically translates or substitute characters.
Syntax:

tr [OPTION] set1 [set2]

Translate, squeeze, and/or delete characters from standard input, writing to standard output.
Options:

c. : complements the set of characters in string.
d. : deletes the characters in set1

s. : replaces repeated characters listed in the set1 with single occurrence
t. : truncates set1

Example: To replace any occurrence of a by x, b by y and c by z in a given string

$echo “about to call “|tr [abc] [xyz]

Output : xyout to zxll

Example: To replace non matching characters

$ echo "Hello"|tr -c e a

Output : aeaaaa

In the above example , except the character “e” other characters are replaced by a

Example: Squeez , we can squeeze more than one occurrence of continuous characters with single occurrence.

$echo “about to call “|tr – s ‘ ‘

Output : about to call

Above example squeezes two or more blank spaces into one.

sort

sort command reorders the lines of a file in ascending or descending order.

The default order is ascending .

Syntax:

sort -t field_delemeter [OPTION] file1 [file 2]

Options:

-k n sort on the nth field of the line

-t char use char as the field delimiter

-n sort numerically

-r reverse order sort

-u removes repeated lines

-m list merge sorted files in list

Examples:

Below examples will help you to understand sort used with different options:

Example 1:

Consider a file named "**list"**, which has below data

1, Justin Timberlake, Title 545, Price $7.30

2, Lady Gaga, Title 118, Price $7.30

3, Johnny Cash, Title 482, Price $6.50

4, Elvis Presley, Title 335, Price $7.30

5, John Lennon, Title 271, Price $7.90

To sort on the 2nd field of file named "list" we have to use the below command:

$sort –t',' –k 2 list

Note: File list is comma separated file.

Output:

4, Elvis Presley, Title 335, Price $7.30

5, John Lennon, Title 271, Price $7.90

3, Johnny Cash, Title 482, Price $6.50

1, Justin Timberlake, Title 545, Price $7.30

2, Lady Gaga, Title 118, Price $7.30

Example 2: Numerically sorting:

To numerically sort data , option to be used is –n

Suppose list is the name of the file having following data:

19

20

5

49

00

If we sort it as below:

$sort list

Output is :

19

20

200

49

5

To get the expected output , the command will be

$sort –n list

Output:

5

19

20

49

200

Sort can sort multiple files also.

$sort file1 file2 file3 …

Example 3: Numerically sort in reverse order

$sort –nr list

Output :

200

49

20

19

5

Example 4: Sort the file list removing the repeated lines.

Syntax:

$sort –u filename

File list has following content:

Unix

Unix

Linux

Linux

Solaris

Axis

Axis

$sort –u list

Output:

Unix

Linux

Solaris

Axis

uniq

uniq command is used to suppress the duplicate lines from a file. It discards all the successive identical lines except one from the input and writes the output.

Syntax:

uniq [option] filename

Options:

-u lists only the lines that are unique

lists only the lines that are duplicates

-c counts the frequency of occurrences

Suppress duplicate lines:

The default behavior of the uniq command is to suppress the duplicate line. Note that, you have to pass sorted input to the uniq, as it compares only successive lines.

If the lines in the file are not in sorted order, then use the sort command and then pipe the output to the uniq command.

Count of lines:

The -c option is used to find how many times each line occurs in the file. It prefixes each line with the count.

Display only duplicate lines:

You can print only the lines that occur more than once in a file using the -d option. The -D option prints all the duplicate lines.

Skip first N fields in comparison:

the -f option is used to skip the first N columns in comparison. Here the fields are delimited by the space character.

cut

This command is used for text processing. You can use this command to extract portion of text from a file by selecting columns.

Syntax:

cut –option filename

Select Column of Characters :

To extract only a desired column from a file use -c option.

The following example displays 2nd character from each line of a file test.txt.

$cut –c2 test.txt

Select Column of Characters using Range :

Range of characters can also be extracted from a file by specifying start and end position delimited with -.

The following example extracts first 3 characters of each line from a file called test.txt

$cut –c 1-3 test.txt

Select Column of Characters using either Start or End Position :

Either start position or end position can be passed to cut command with -c option.

Following example extracts from 3rd character to end of each line from test.txt file.

$cut –c3- test.txt

To extract 8 characters from the beginning from the file test.txt,

$cut –c-8 test.txt

Select a Specific Field from a File :

Instead of selecting x number of characters you can combine option -f and –d to extract a whole field.

The option -f specifies which field you want to extract,

The option -d specifies what delimiter that is used in the input file.

The following example displays only first field of each lines from /etc/passwd file using the field delimiter**:** (colon). In this case, the 1st field is the username.

$ cut -d':' -f1 etc/passwd

paste

This is the command for merging together different files into a single, multi-column file. In combination with cut, useful for creating system log files.

Syntax:

paste file1 file2

join

This utility allows merging two files in a meaningful fashion, which essentially creates a simple version of a relational database.

The command join operates on exactly two files, but pastes together only those lines with a common tagged field (usually a numerical label), and writes the result to standard output.

The files to be joined should be sorted according to the tagged field for the matchups to work properly.

Example:

The content of two files file1 and file2 are as below,
$cat file1

100 Shoes

200 Laces

300 Socks

$cat file2

100 $40.0

200 $1.00

300 $2.00

The following command will join these two files.

$ join 1.data 2.data

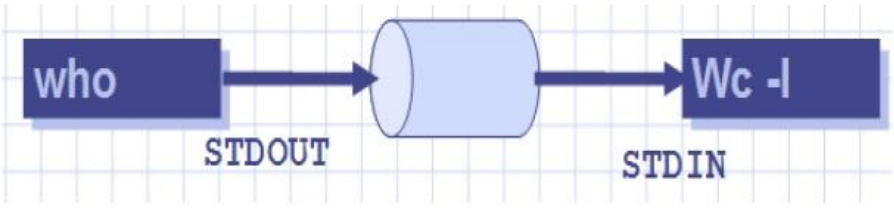
100 Shoes $40.00

200 Laces $1.00

300 Socks $2.00

Pipe

In unix , you can connect two commands together so that the output from one program becomes the input of the next program. Two or more commands connected in this way form a pipe. In shell the symbol '|' is used to represent pipe.



Example:

```
$ who | wc –l
```

Purpose of Pipes :

Using pipe you can construct powerful unix command lines by combining basic unix commands. UNIX commands are powerful; however by using pipe you can combine them together, to accomplish complex tasks with ease.

Through the standard output of one command (the command to the left of the pipe) gets sent as standard input to another command (the command to the right of the pipe). Pipe functions in a similar manner like the output redirection in UNIX (using > symbol to redirect the standard output of a command to a file. However, the pipe is different because it is used to pass the output of a command to another command, not a file.

Example:

$ cat apple.txt | wc

3 4 21
In this example, the contents of the file apple.txt are sent through pipe to wc (word count) command. The wc command then does its job and counts the lines, words, and characters in the file.

You can combine many commands with pipes on a single command line. Here's an example where the characters, words, and lines of the file apple.txt is sent to wc and then the output of wc mailed to nobody@december.com with the subject line "The count."

$ cat apple.txt | wc | mail -s "The count" nobody@december.com

Filters

Filter is a programm in unix . It takes its input from another program, performs some operation on that input, and writes the result to the standard output. Thus the common use of filters is to modify or restructure output.

Some common filters in UNIX are:

- uniq – Removes identical adjacent lines
- head – displays first n lines of a file .
- tail – displays last n lines of a file .
- sort – sorts files by line (lexically or numerically)
- cut – select portions of a line.
- wc – word count (line count, character count)
- tr – translate
- grep, egrep – search files using regular expressions

head

This command list the beginning of a file to standard output. The default is 10 lines, but a different number can be specified. The command has a number of options.

Syntax:

head [OPTION] [FILE]

Options:

-c Prints the first N bytes of file; with leading -, prints all but the last N bytes of the file.

-n Prints first N lines; with leading - print all but the last N lines of each file.

Example: To display the first 10 lines of the file myfile.txt. $head myfile.txt

To display the first 100 lines of the file myfile.txt.

$head -n100 myfile.txt

To print the first 5 bytes from the file

$ head -c5 myfile.txt

tail

List the (tail) end of a file to stdout. The default is 10 lines, but this can be changed with the -n option. Commonly used to keep track of changes to a system log-file, using the -f option, which outputs lines appended to the file.

Syntax:

tail [OPTION]... [FILE]...

Example:

To display the last 10 lines of the file myfile.txt.

$tail myfile

To display the last 100 lines of the file myfile.txt.

$ tail -100 myfile.txt

$tail –n 100 myfile.txt

more

more command allows to view text files or other output in a scrollable manner. When can command is used to view a very long file, all the output scrolls off the top of your screen and only the last page can be viewed. more command solves this problem by allowing the output of cat command one screenful of data at a time.

Syntax:

more [option] filename

Options:

-num This option specifies an integer which is the screen size (in lines).

-d more will prompt the user with the message "[Press space to continue, 'q' to quit.]" and will display "[Press 'h' for instructions.]" instead of ringing the bell when an illegal key is pressed.

-l more usually treats ^L (form feed) as a special character, and will pause after any line that contains a form feed. The -l option will prevent this behavior.

-p Do not scroll. Instead, clear the whole screen and then display the text.

tr

tr command automatically translates or substitute characters.

Syntax:

tr [OPTION] set1 [set2]

Translate, squeeze, and/or delete characters from standard input, writing to standard output.

Options:

c. : complements the set of characters in string.
d. : deletes the characters in set1

s. : replaces repeated characters listed in the set1 with single occurrence
t. : truncates set1

Example: To replace any occurrence of a by x, b by y and c by z in a given string

$echo “about to call “|tr [abc] [xyz]

Output : xyout to zxll

Example: To replace non matching characters

$ echo "Hello"|tr -c e a

Output : aeaaaa

In the above example , except the character “e” other characters are replaced by a

Example: Squeez , we can squeeze more than one occurrence of continuous characters with single occurrence.

$echo “about to call “|tr – s ‘ ‘

Output : about to call

Above example squeezes two or more blank spaces into one.

sort

sort command reorders the lines of a file in ascending or descending order.

The default order is ascending .

Syntax:

sort -t field_delemeter [OPTION] file1 [file 2]

Options:

-k n sort on the nth field of the line

-t char use char as the field delimiter

-n sort numerically

-r reverse order sort

-u removes repeated lines

-m list merge sorted files in list

Examples:

Below examples will help you to understand sort used with different options:

Example 1:

Consider a file named "**list"**, which has below data

1, Justin Timberlake, Title 545, Price $7.30

2, Lady Gaga, Title 118, Price $7.30

3, Johnny Cash, Title 482, Price $6.50

4, Elvis Presley, Title 335, Price $7.30

5, John Lennon, Title 271, Price $7.90

To sort on the 2nd field of file named "list" we have to use the below command:

$sort –t',' –k 2 list

Note: File list is comma separated file.

Output:

4, Elvis Presley, Title 335, Price $7.30

5, John Lennon, Title 271, Price $7.90

3, Johnny Cash, Title 482, Price $6.50

1, Justin Timberlake, Title 545, Price $7.30

2, Lady Gaga, Title 118, Price $7.30

Example 2: Numerically sorting:

To numerically sort data , option to be used is –n

Suppose list is the name of the file having following data:

19

20

5

49

00

If we sort it as below:

$sort list

Output is :

19

20

200

49

5

To get the expected output , the command will be

$sort –n list

Output:

5

19

20

49

200

Sort can sort multiple files also.

$sort file1 file2 file3 …

Example 3: Numerically sort in reverse order

$sort –nr list

Output :

200

49

20

19

5

Example 4: Sort the file list removing the repeated lines.

Syntax:

$sort –u filename

File list has following content:

Unix

Unix

Linux

Linux

Solaris

Axis

Axis

$sort –u list

Output:

Unix

Linux

Solaris

Axis

uniq

uniq command is used to suppress the duplicate lines from a file. It discards all the successive identical lines except one from the input and writes the output.

Syntax:

uniq [option] filename

Options:

-u lists only the lines that are unique

lists only the lines that are duplicates

-c counts the frequency of occurrences

Suppress duplicate lines:

The default behavior of the uniq command is to suppress the duplicate line. Note that, you have to pass sorted input to the uniq, as it compares only successive lines.

If the lines in the file are not in sorted order, then use the sort command and then pipe the output to the uniq command.

Count of lines:

The -c option is used to find how many times each line occurs in the file. It prefixes each line with the count.

Display only duplicate lines:

You can print only the lines that occur more than once in a file using the -d option. The -D option prints all the duplicate lines.

Skip first N fields in comparison:

the -f option is used to skip the first N columns in comparison. Here the fields are delimited by the space character.

cut

This command is used for text processing. You can use this command to extract portion of text from a file by selecting columns.

Syntax:

cut –option filename

Select Column of Characters :

To extract only a desired column from a file use -c option.

The following example displays 2nd character from each line of a file test.txt.

$cut –c2 test.txt

Select Column of Characters using Range :

Range of characters can also be extracted from a file by specifying start and end position delimited with -.

The following example extracts first 3 characters of each line from a file called test.txt

$cut –c 1-3 test.txt

Select Column of Characters using either Start or End Position :

Either start position or end position can be passed to cut command with -c option.

Following example extracts from 3rd character to end of each line from test.txt file.

$cut –c3- test.txt

To extract 8 characters from the beginning from the file test.txt,

$cut –c-8 test.txt

Select a Specific Field from a File :

Instead of selecting x number of characters you can combine option -f and –d to extract a whole field.

The option -f specifies which field you want to extract,

The option -d specifies what delimiter that is used in the input file.

The following example displays only first field of each lines from /etc/passwd file using the field delimiter**:** (colon). In this case, the 1st field is the username.

$ cut -d':' -f1 etc/passwd

paste

This is the command for merging together different files into a single, multi-column file. In combination with cut, useful for creating system log files.

Syntax:

paste file1 file2

join

This utility allows merging two files in a meaningful fashion, which essentially creates a simple version of a relational database.

The command join operates on exactly two files, but pastes together only those lines with a common tagged field (usually a numerical label), and writes the result to standard output.

The files to be joined should be sorted according to the tagged field for the matchups to work properly.

Example:

The content of two files file1 and file2 are as below,

$cat file1

100 Shoes

200 Laces

300 Socks

$cat file2

100 $40.0

200 $1.00

300 $2.00

The following command will join these two files.

$ join 1.data 2.data

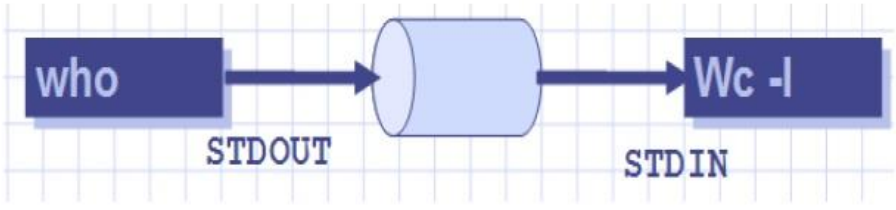
100 Shoes $40.00

200 Laces $1.00

300 Socks $2.00

Pipe

In unix , you can connect two commands together so that the output from one program becomes the input of the next program. Two or more commands connected in this way form a pipe. In shell the symbol '|' is used to represent pipe.



Example:

```
$ who | wc –l
```

Purpose of Pipes :

Using pipe you can construct powerful unix command lines by combining basic unix commands. UNIX commands are powerful; however by using pipe you can combine them together, to accomplish complex tasks with ease.

Through the standard output of one command (the command to the left of the pipe) gets sent as standard input to another command (the command to the right of the pipe). Pipe functions in a similar manner like the output redirection in UNIX (using > symbol to redirect the standard output of a command to a file. However, the pipe is different because it is used to pass the output of a command to another command, not a file.

Example:

$ cat apple.txt | wc

3 4 21
In this example, the contents of the file apple.txt are sent through pipe to wc (word count) command. The wc command then does its job and counts the lines, words, and characters in the file.

You can combine many commands with pipes on a single command line. Here's an example where the characters, words, and lines of the file apple.txt is sent to wc and then the output of wc mailed to nobody@december.com with the subject line "The count."

$ cat apple.txt | wc | mail -s "The count" nobody@december.com